Professor Demo Cheat Sheet

Keep this beside you during the presentation. The detailed explanations are in DEMO-AND-VIVA.md.

Before you start

Keep EC2 running. Open the website and the GitHub Actions page. Confirm the latest run is green. You do not need to reconnect SSH just to show a code change.

What to say

“I built a Java Spring Boot task tracker. GitHub Actions tests it using Maven. The tested JAR is packaged into Docker, pushed to Amazon ECR and deployed to K3s on EC2 using Ansible. Health checks verify that the release is ready.”

Show a change in six steps

1. Open github.com/MohithR2005/devops_final_lab.
2. Open src/main/resources/application.properties and click the pencil.
3. Change app.version from 1.0.1 to 1.0.2, or to the next number if already changed.
4. Commit directly to main with a clear message.
5. Open Actions and wait for build-test and deploy to turn green.
6. Refresh http://13.236.232.232 with Ctrl+F5 and show the new badge.

While the pipeline runs

Explain that CI compiles and tests on GitHub. The deploy job uses the tested artifact, builds the image on EC2, pushes ECR and applies Kubernetes YAML through Ansible. No manual build, image push or kubectl deployment is needed for the demo.

Remember

The update can take several minutes and briefly interrupt the website. Tasks reset when the Java pod is replaced. Closing your laptop or SSH session does not erase the setup. A changed hotspot only affects the IP allowed for SSH. If EC2 was stopped/started, check its new public IP.

Viva answers to remember

Term	Simple explanation
CI / CD	CI builds and tests changes; CD automatically releases successful changes to the running application.
Maven	Build tool that resolves dependencies, compiles Java, runs tests and packages the JAR.
Git / GitHub	Git tracks versions; GitHub stores the repository and runs the automation.
Docker image / container	Image is the packaged template; container is a running instance.
ECR / EC2	ECR stores images; EC2 is the virtual server running the cluster and deployment runner.
K3s / Kubernetes	K3s is the lightweight Kubernetes distribution used to manage the application pods.
Pod / Deployment	Pod runs the container; Deployment maintains and updates the desired pod count.
Service / Ingress	Service gives pods a stable internal address; Ingress routes external HTTP requests through Traefik.
Ansible	Applies repeatable configuration: ECR renewal timer, image replacement, Kubernetes apply and checks.
IAM role	Gives the EC2 server temporary AWS permissions without storing access keys in GitHub.
Probes	Startup allows boot time; readiness controls traffic; liveness detects a container needing restart.
Why one replica?	Small server and in-memory task list. Recreate uses less memory but causes brief update downtime.
Why two Dockerfiles?	Local builds compile from source; CI runtime builds use the already tested JAR to save server resources.
What if tests fail?	Deployment is skipped and the current release remains running.
Production limitations	No database, TLS, login or high availability; the runner has administrator privileges.

Useful commands if asked

```
sudo kubectl get nodes
sudo kubectl get pods,svc,ingress -n final-lab
sudo kubectl logs deployment/final-lab -n final-lab --tail=20
curl -s http://localhost/actuator/health
```

Do not claim ECS or EKS were used: this project uses K3s Kubernetes on an ordinary EC2 instance.